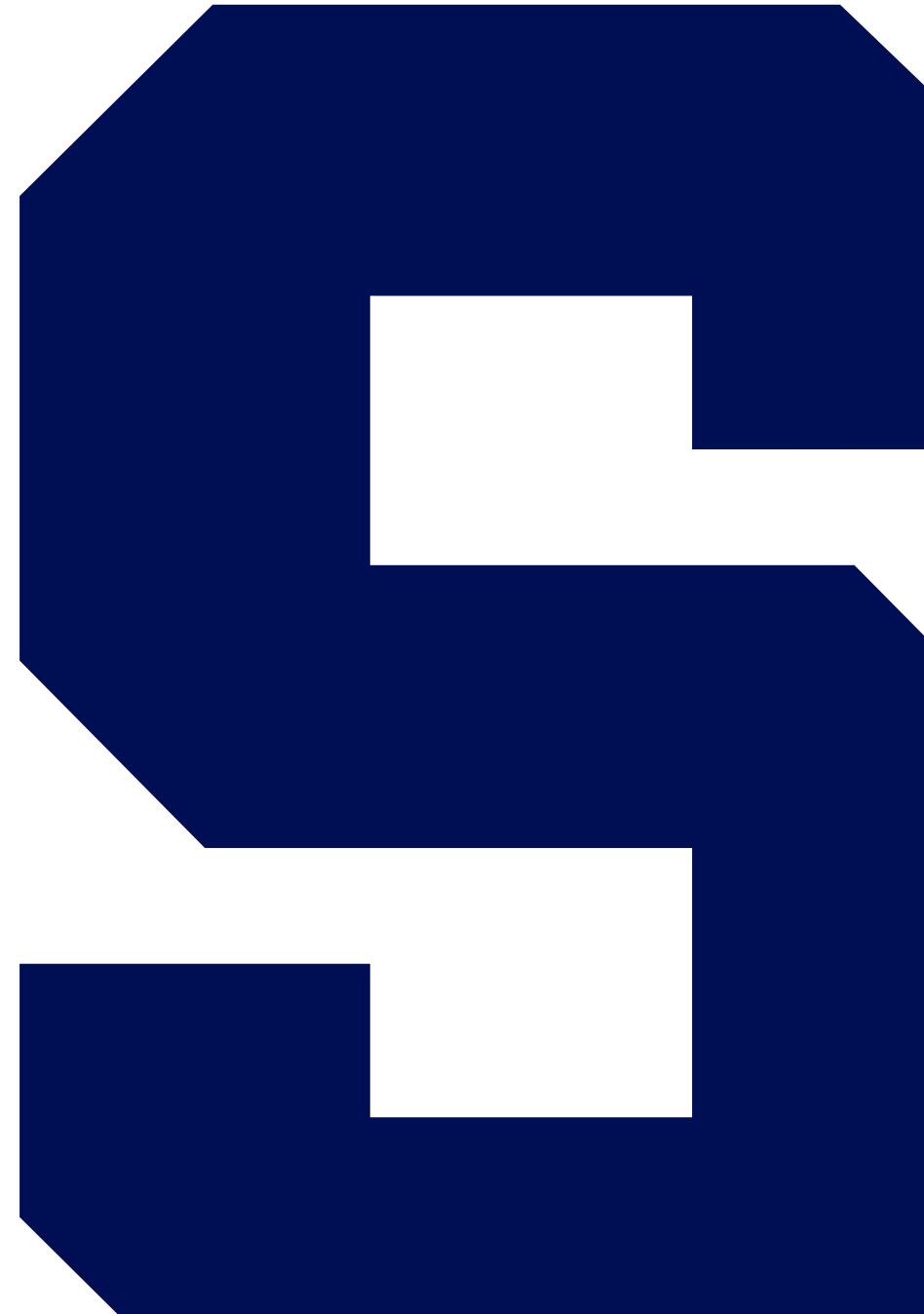


Dimensionality Reduction Overview



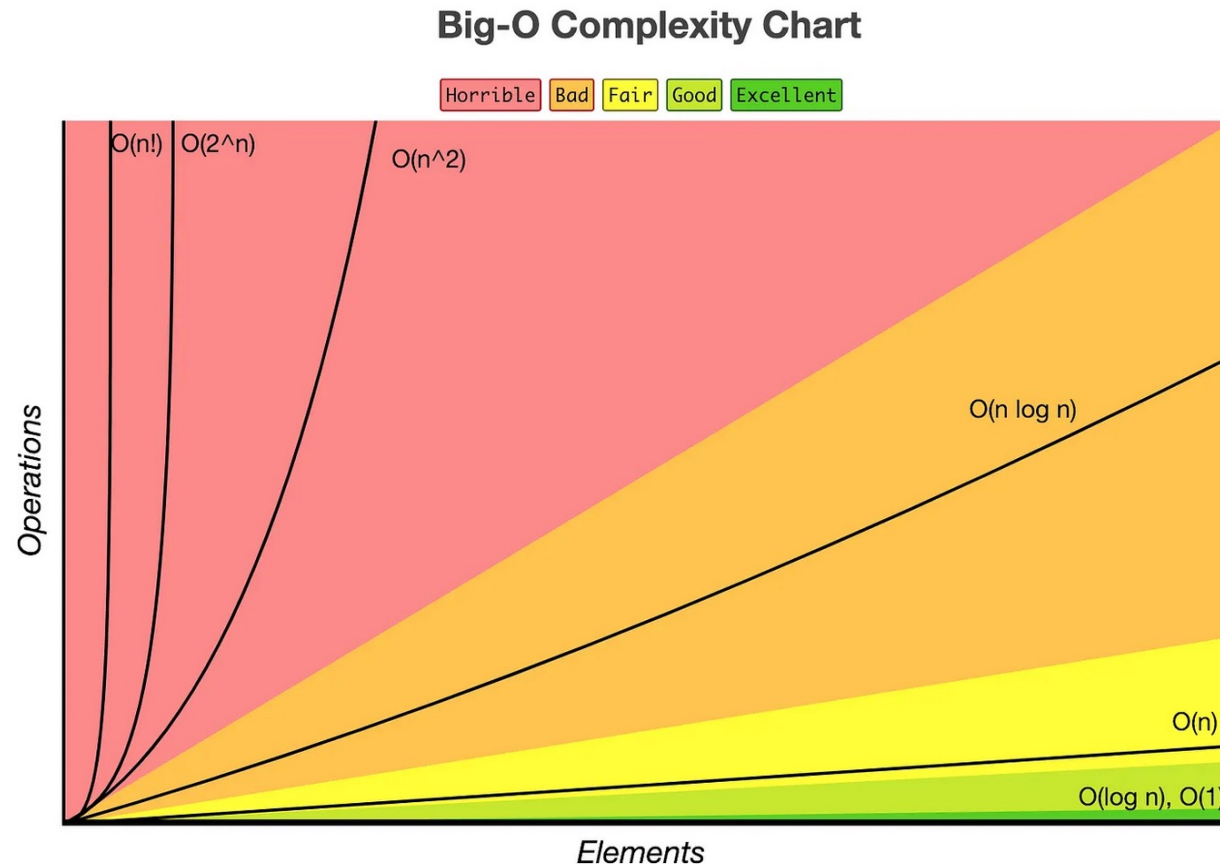
Dimensionality Reduction

- A technique used in ML and statistics to **reduce the number of input variables in a dataset**.
 - More input features make models more complex and less interpretable, and risk overfitting.
 - Dimensionality reduction techniques aim to **simplify models** without losing much information.
-

Why is Dimensionality Reduction Important?

1. **Simplification:** Makes the dataset easier to explore and visualize.
 2. **Speed:** Reduces the computational complexity, making algorithms run faster.
 3. **Data Compression:** Allows for more efficient storage of data.
 4. **Noise Reduction:** Helps to eliminate irrelevant features or reduce noise.
 5. **Improved Performance:** Can lead to better model performance when irrelevant features are removed.
-

Big-O Notation and Algorithmic Speed



Algorithm	Training Time
SVM	$O(m \cdot n^2) - O(m \cdot n^3)$
Random Forest	$O(m \cdot n \cdot \log(n) \cdot k)$
Decision Trees	$O(m \cdot n \cdot \log(n))$
Logistic Regression	$O(m \cdot n)$
KNN *	$O(k \cdot n \cdot \min(m, n))$

m = features

n = training samples

k = hyperparameters

* pre-processing

Dimensionality Reduction Example

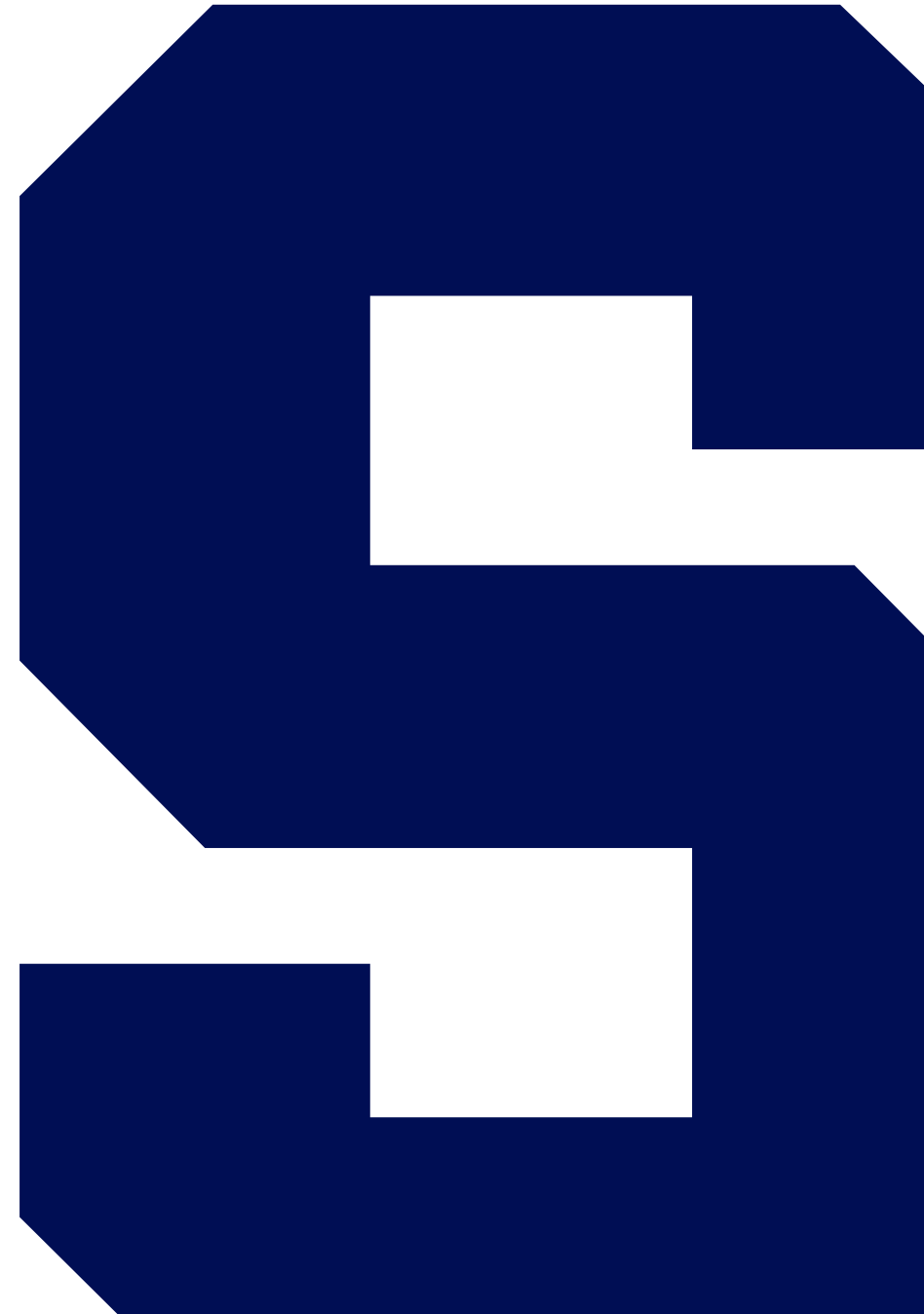
Context: Imagine you're a data scientist at a retail company. The company collects data on customer transactions, online activity, customer service interactions, and more. In total, you have hundreds of features for each customer.

Problem: You want to segment your customer base to target marketing more effectively, but the high dimensionality of your data makes it difficult to gain any meaningful insights.

Solution: By applying dimensionality reduction techniques like PCA, you can reduce your hundreds of features to just a few principal components. These components can effectively summarize the essential information in the dataset, making it easier to perform customer segmentation.

Outcome: The marketing team can now more effectively tailor strategies for different segments, thereby increasing customer engagement and revenue.

The Curse of Dimensionality



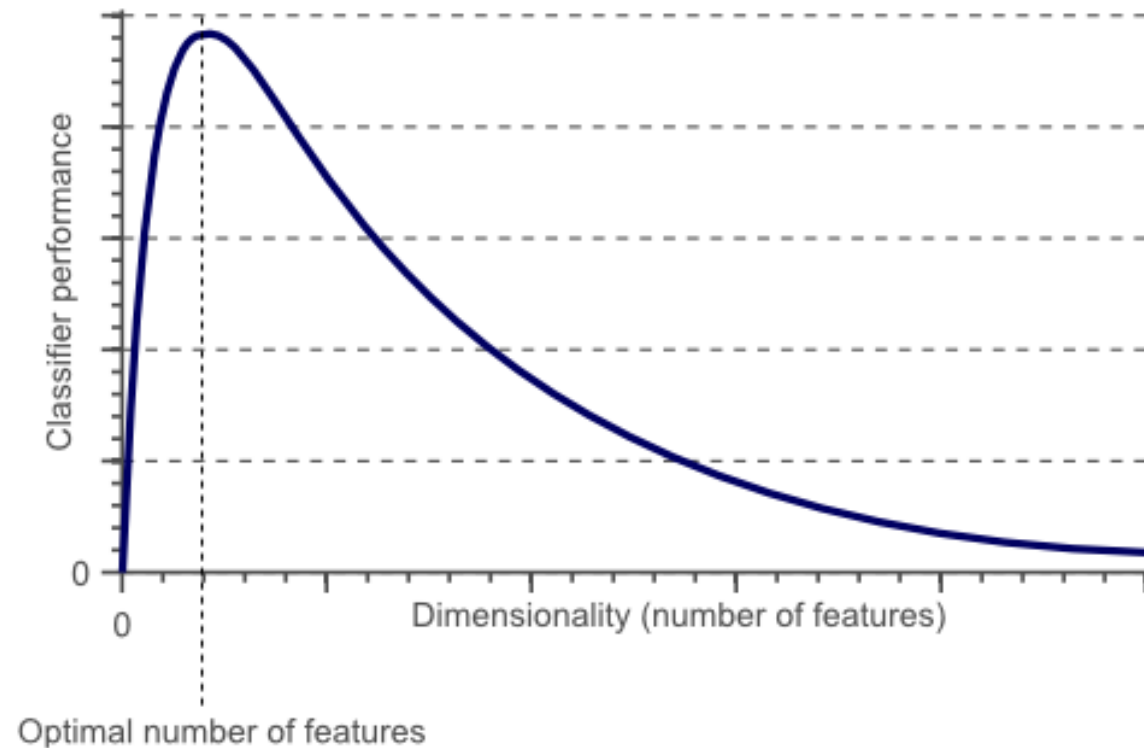
The Curse of Dimensionality

In high dimensions, the behavior of data can be counterintuitive, and algorithms that work well in low dimensions can become ineffective or computationally expensive. This is because the available "space" that training samples are embedded grows exponentially with respect to the data. This has several ramifications:

- Distances between items become larger.
 - Differences between distances becomes smaller.
 - Amount of data required to cover the space grows exponentially.
-

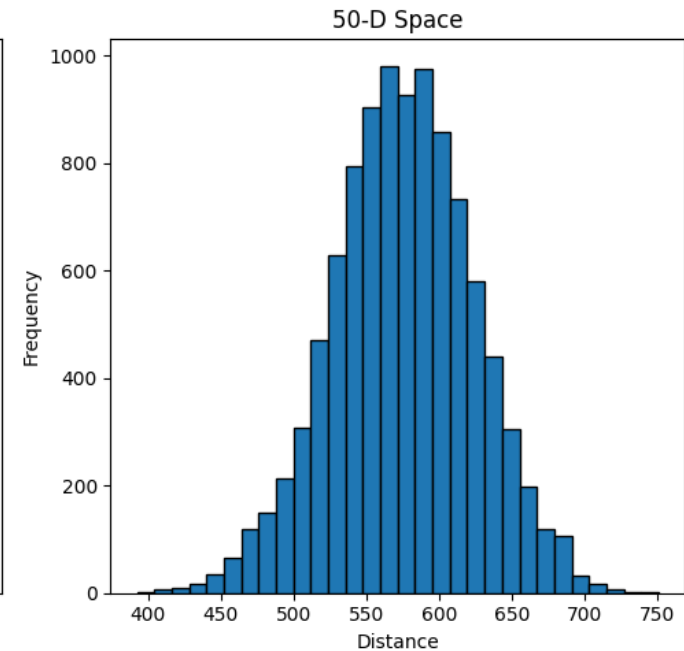
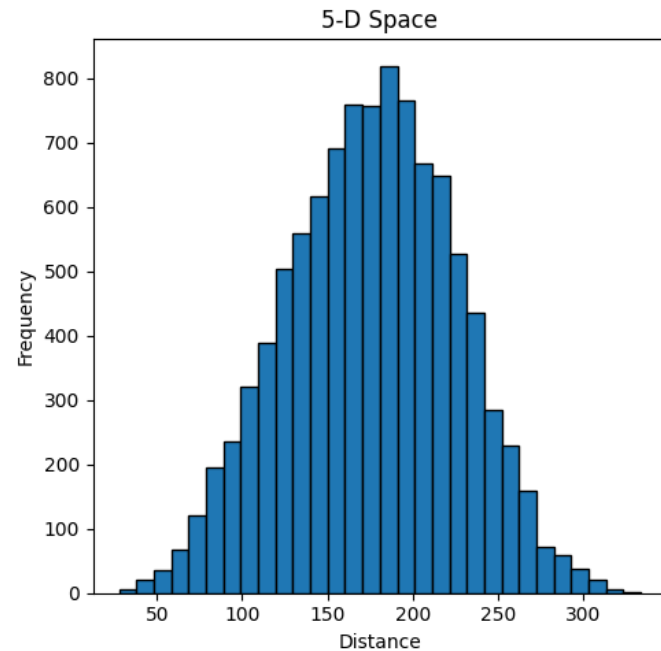
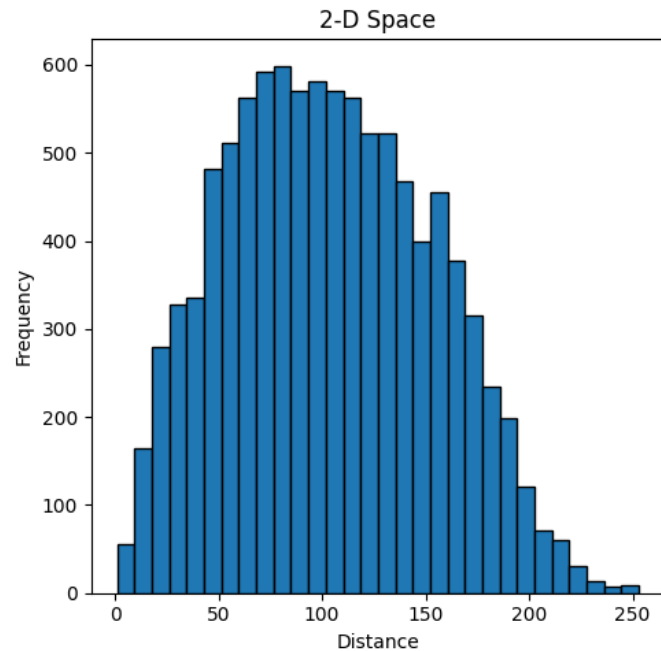
The Hughes Phenomenon

With a fixed number of training samples, the average (expected) predictive power of a classifier or regressor first increases as the number of dimensions (features) used is increased but beyond a certain dimensionality it starts deteriorating instead of improving steadily.



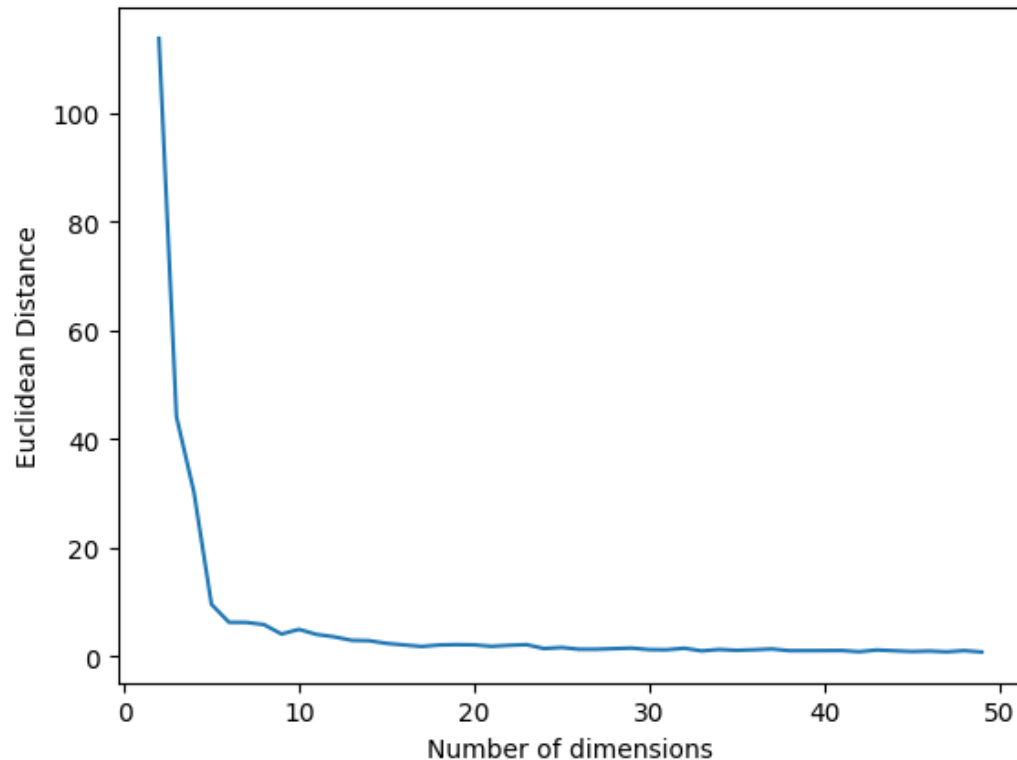
Higher Dimensions Equals Greater Distance

As the number of dimensions increases, the average pairwise Euclidian distance between points increases. This sparsity means that a larger amount of data is required to adequately cover the space.



Higher Dimensions Equals Smaller Differences

Calculating the ratio of the range of Euclidean distances to the minimum distance of 10,000 random points to a random fixed point, across various dimensions.



As the number of dimensions increases, the relative difference in distances between the maximum and minimum distances to a fixed point becomes smaller.

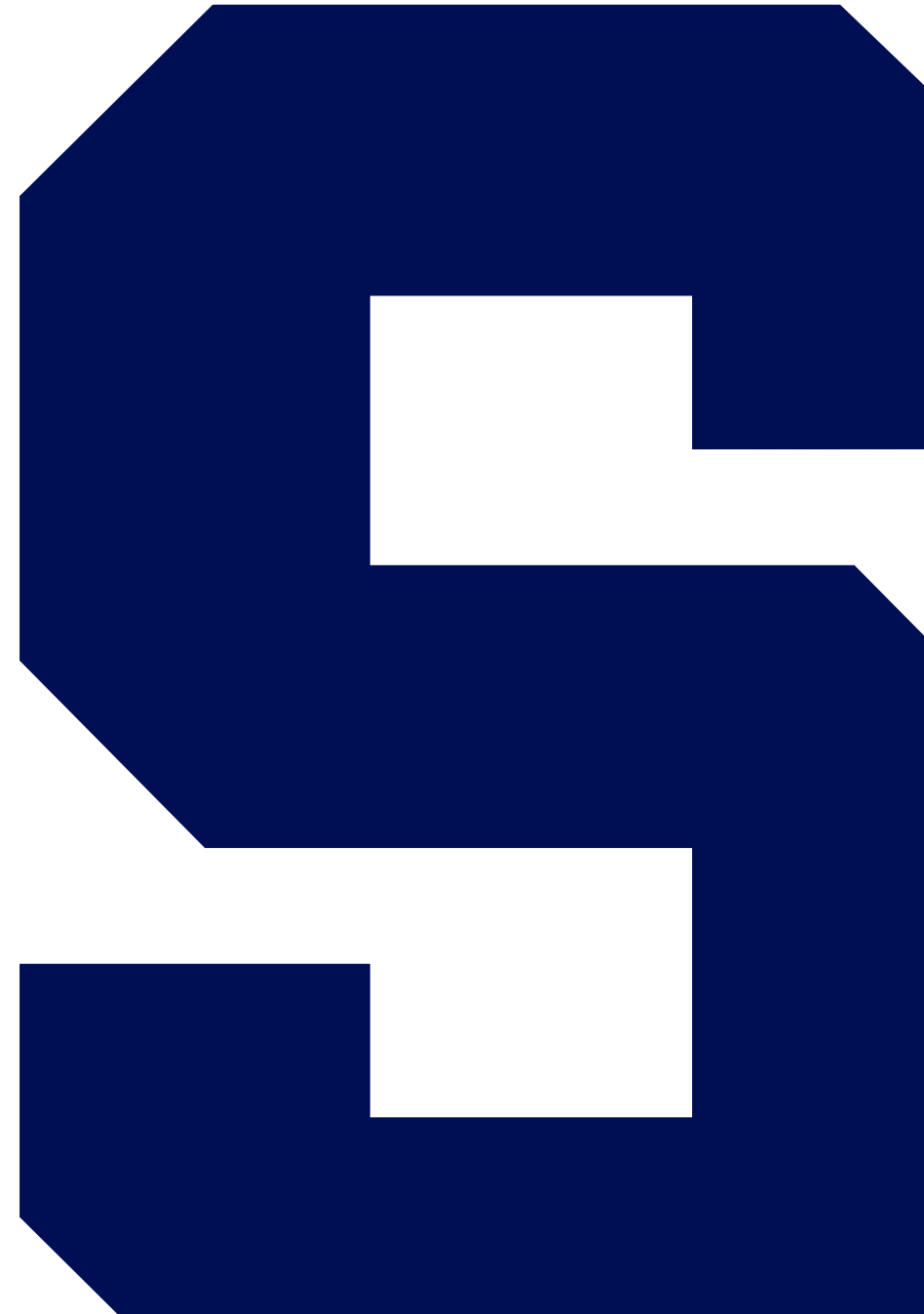
In other words, in high-dimensional spaces, **all points tend to become equidistant from each other**, making it harder to discern differences in distances, which complicates tasks like clustering and nearest-neighbor calculations.

What to Do?

Dimensionality Reduction Techniques:

- Principal Component Analysis (PCA)
 - t-Distributed Stochastic Neighbor Embedding (t-SNE)
 - Uniform Manifold Approximation and Projection (UMAP)
-

Principal Component Analysis (PCA)

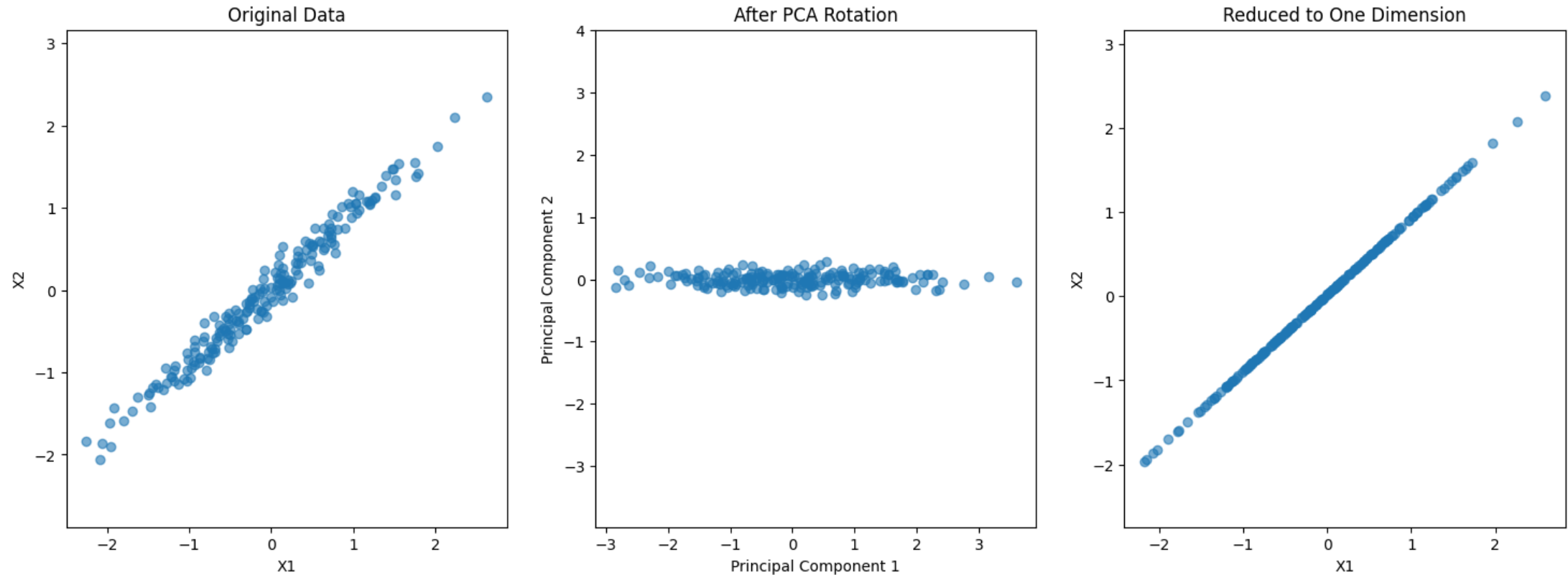


Principal Component Analysis

- A statistical technique used for dimensionality reduction that transforms the original features of a data set to a **new set of features**.
 - The new features (**principal components**) are linear combinations of the original features of the dataset.
 - PCA works by performing an eigenvector/eigenvalue decomposition of the data's covariance matrix.
 - Principal components are **uncorrelated** with each other.
 - The first few principal components typically **capture the majority of the variance** of a dataset.
-

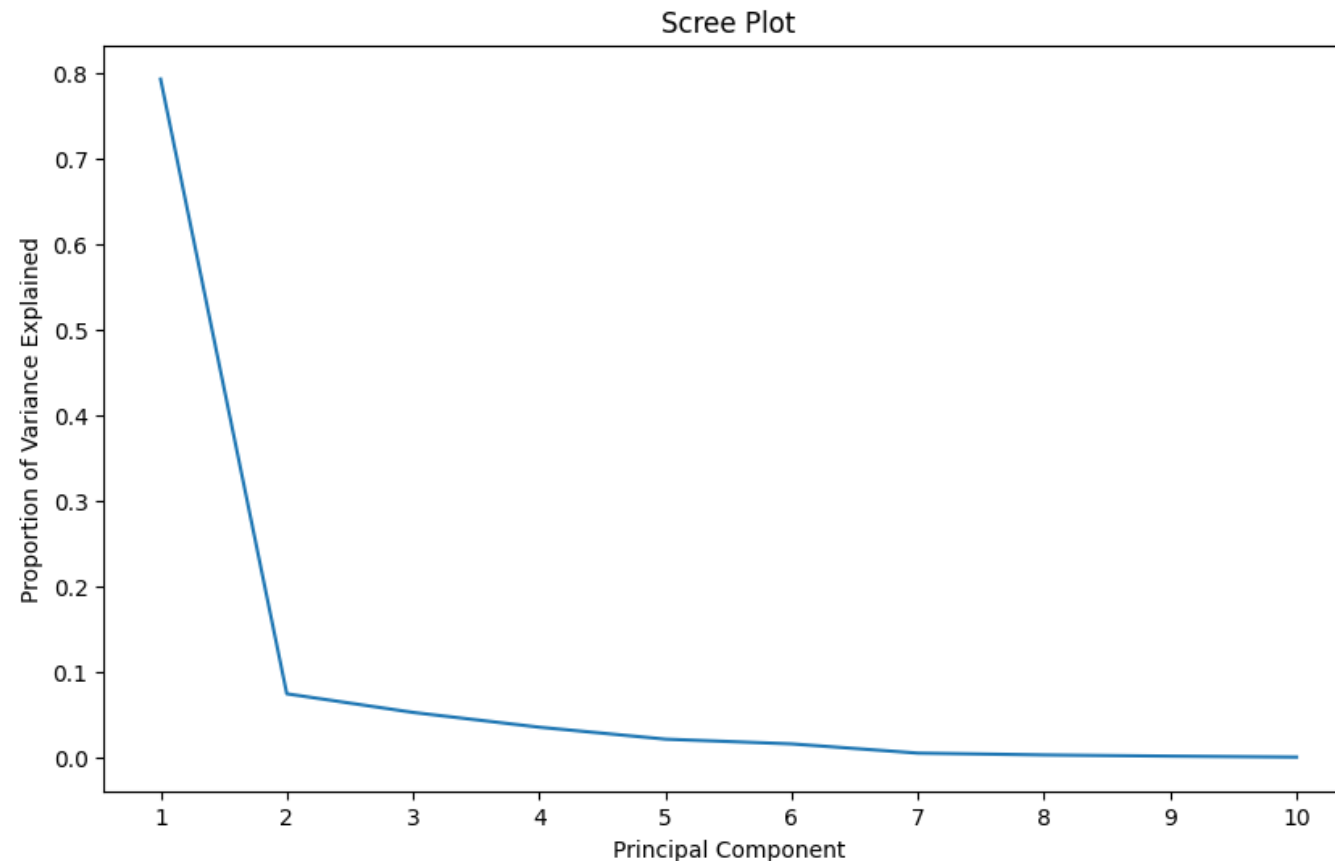
PCA Illustration

We are able to capture the essential information of a two-dimensional dataset using only a single dimension.



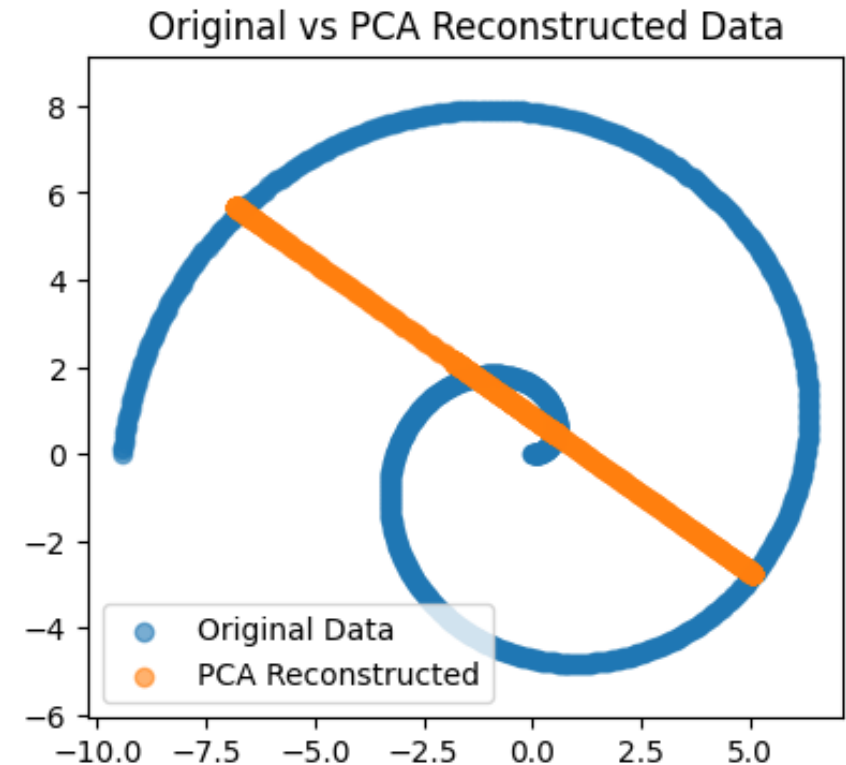
Selecting the Right Number of Components

Scree plots help to identify the number of principal components needed to capture a certain amount of variance in the dataset.



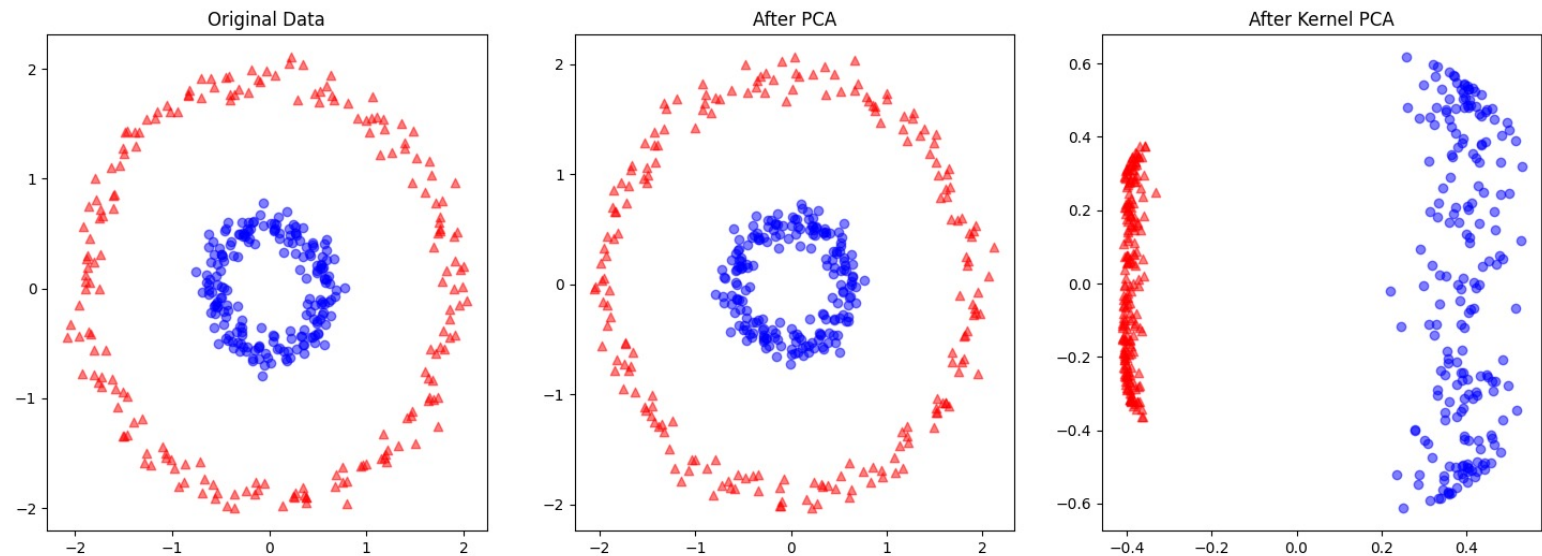
Limitations of PCA

- PCA is a linear technique (i.e., a linear combination of original features).
- PCA is not suitable for capturing patterns in non-linear data.



Alternative PCA Approaches

Several extensions to PCA address its limitations and extend its capabilities. Kernel Principal Component Analysis (**Kernel PCA**) is an extension of the standard Principal Component Analysis (PCA), designed to handle **non-linear** data effectively.



Kernel PCA: Advantages and Disadvantages

While standard PCA works by finding linear principal components to project data onto, Kernel PCA works in an implicitly defined higher-dimensional feature space where these projections can be non-linear.

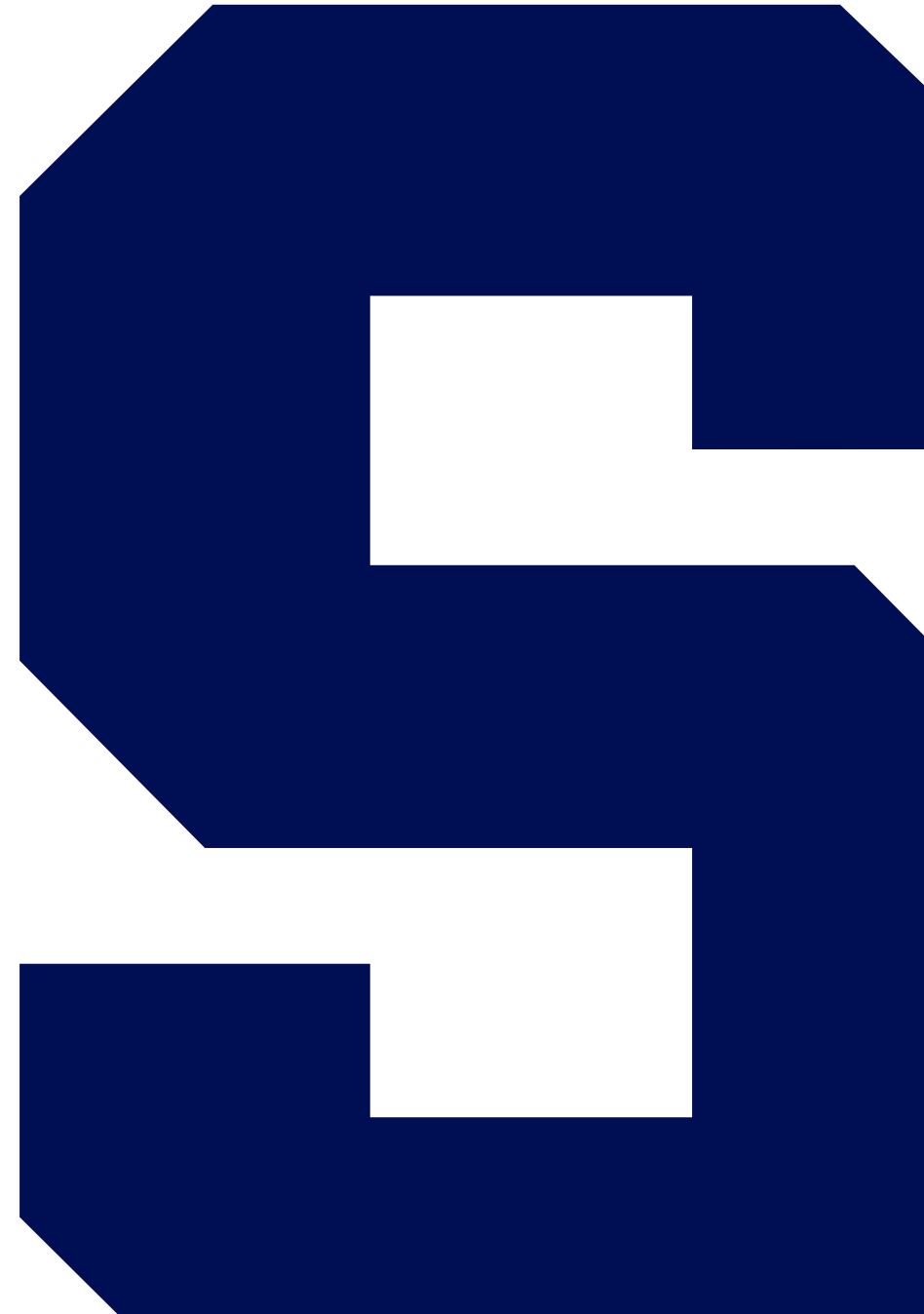
Advantages:

- Can capture complex, non-linear relationships in data.
- Suitable for clustering, classification, and other tasks where linear methods fail.

Disadvantages:

- Computationally more expensive compared to standard PCA.
 - Requires choosing an appropriate kernel function and tuning its parameters, which may not be straightforward.
-

Adaptive Techniques: t-SNE and UMAP



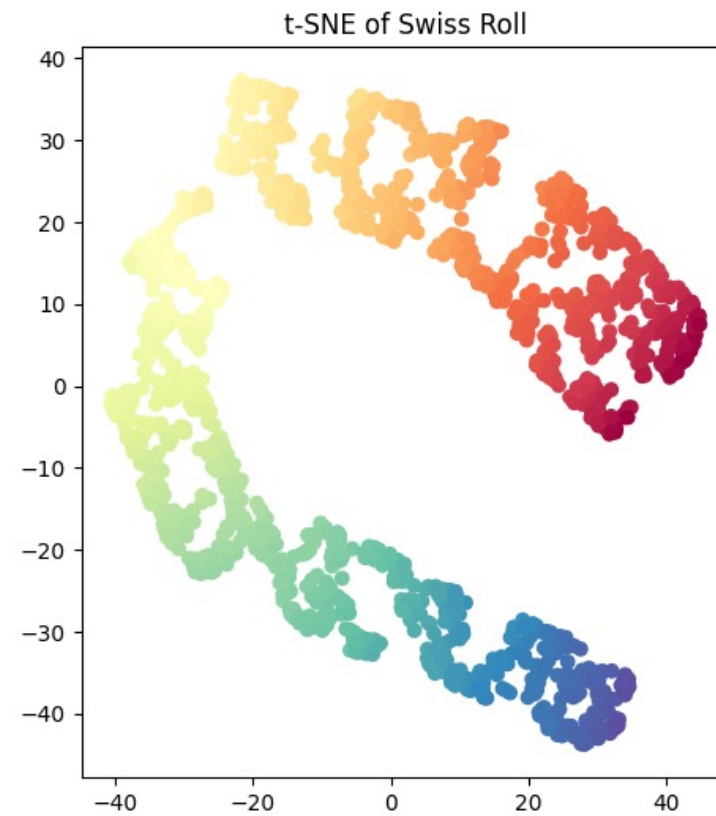
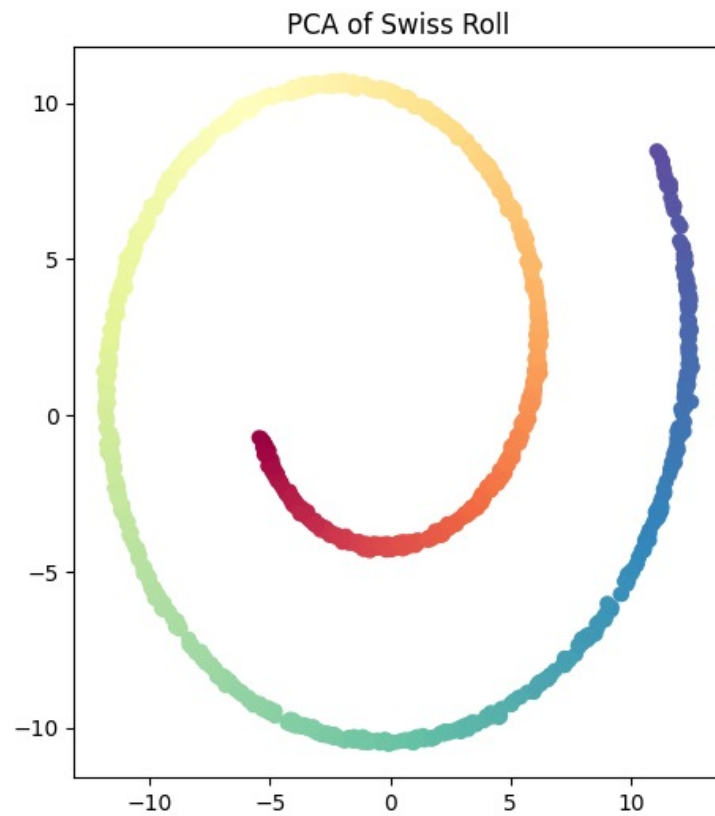
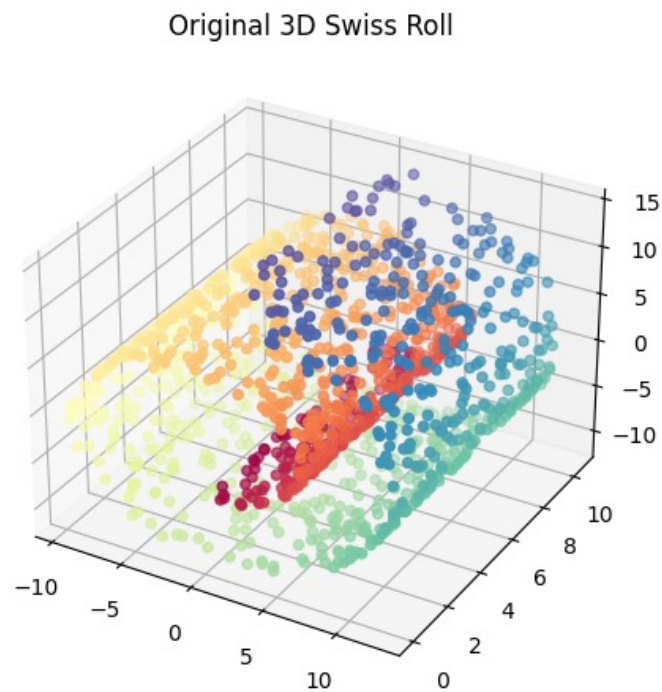
Adaptive Techniques

- Adaptive techniques preserve complex, **non-linear** relationships between data points.
 - Instead of applying the same transformation to all data points equally (e.g., PCA), adaptive methods optimize transformations for different regions of the feature space.
 - This feature allows these methods to **preserve complex structures** such as clusters and manifolds in high-dimensional space.
 - However, they are **stochastic** and not guaranteed to preserve distances in the transformed data.
 - Examples: **t-SNE** (t-Distributed Stochastic Neighbor Embedding, 2002), **UMAP** (Uniform Manifold Approximation and Projection, 2018)
-

t-SNE Overview

- Particularly well-suited for **visualization** of high-dimensional data.
 - t-SNE converts similarities between data points to joint probabilities and tries to **minimize the Kullback-Leibler divergence** between the joint probabilities of the low-dimensional embedding and the high-dimensional data.
 - In other words, t-SNE aims to ensure that data points that are neighbors in high-dimensional space will continue to be neighbors in a lower-dimensional space.
 - Good at **preserving local structures** but may not preserve global structures.
 - Computationally intensive.
-

t-SNE Example



UMAP Overview

- UMAP is a **graph-based approach**: UMAP first constructs a weighted k-NN graph for the data in the original space. It then optimizes a similar graph in the lower-dimensional space to be as structurally similar to the original graph as possible.
 - UMAP aims to preserve not just local but also **global data structures**.
 - UMAP is designed to be **computationally efficient**, making it a more scalable option than t-SNE.
 - UMAP is not just for visualization; you can also use it for **general** non-linear dimensionality reduction tasks.
-

UMAP Comparison to t-SNE

